

Speedup-search follow-up: what ships from the AVW thin-product thread

Date: 2026-10-06. Status: complete for the eight items the follow-up report's *Decision* section lists as shippable. Every item is labelled **checked** (exact algebra verified by code), **derived** (follows from the stated premise by calculation), or **measured** (a stage diagnostic on an ordinary shared host). Nothing here is an IC1 result; every measured row keeps `candidate_id`: null.

Report PDF: `speedup_search_followup.pdf` (built by `build_pdf.py` from this file's content and the figures).

Diagram: `figures/decision_map.svg` (source `figures/make_figures.py`).

Question and scope

The follow-up report ("Speedup Search Follow-up: Collision-generated relations, batch-smoothness barriers, and growing-arity elliptic curves", 2026-10-06, uploaded by the user) asked whether the *selected thin product* theorem it cites as AVW (arXiv:2610.06783; the "AVW premise") yields any faster NFS, FFS, ECDLP, pairing-target, or complete index-calculus algorithm, and concluded that it does not, while listing eight pieces that can be shipped as reusable lemmas, screens and constructions. This directory implements and checks those eight pieces.

Premise as used throughout (`avw_model.py`): with an $N \times D$ by $D \times N$ product and w selected entries, AVW costs $N^2 / D^{0.063}$ preprocessing plus $D^{0.437}$ per query, requires $D \leq N^{(1/18)}$, and needs separation rank $s > n^{0.437}$ of the inner dimension n .

The claim boundary is the report's own and is restated for every item: **no faster NFS, FFS, ECDLP, pairing-target attack, or complete index-calculus algorithm is claimed from AVW.**

Speedup-search follow-up (2026-10-06): what ships, with status

Premise: AVW selected thin products, $D \leq N^{1/18}$, preprocessing $N^{2/D^{0.063}}$, query $D^{0.437}$, separation rank $> n^{0.437}$

■ checked algebra ■ derived from premise ■ measured (stage diagnostic)

1 Selected-resieve correspondence selected sieve scores = selected entries of XY, inner dim $D = \sum p \leq y$; rank $K = 1 + \sum(p-1)$: checked on 5 prime sets, 40 identity trials; $D \leq N^{1/18}$ caps y at 5 ($N=2^{64}$), 29 (2^{128}), 461 (2^{256})	lemma
2 Batch-smoothness screen product/remainder-tree smooth parts, 900 values checked vs trial division; ~25-37 ns per bit of the batch, flat in y and W ; dominates AVW by $D^{0.437}$ inside the preprocessing range (derived)	screen
3 NFS density window $0.437g < \theta < 0.563g$; $g \leq 1/18$ gives $0.02428 < \theta < 0.03128$; GNFS $\theta=1/4$ and SNFS $\theta=1/2$ fall outside; AVW $S^{1.9965}$ vs batch $S^{1.75} / S^{1.5}$	no-go
4 Dyadic heavy-cell reporter exact for nonnegative scores; queries $\leq 1 + 4(M/\tau) \log_2 N$ on 25 random + 16 sieve instances; NFS screen: $M/\tau = N^{(2-o(1))}$, never beats a plain scan	construction
5 SSS five-fixtue positive control 15/15 verified; SSS 0.519 s vs pSIQS 3.582 s (GM 6.58x) vs SymPy SIQS 8.111 s (GM 12.87x); 30-digit N, shared VM	benchmark
6 Joux advanced pinpointing toy Kummer join $q=311$, $n=5$: 620 relations verified; $L(1/3)$ constants: range $[3^{2/3}, (2/3)^{2/3}]$, 1.4422 $\rightarrow$ 0.9615 at α_{low} (paper misprint 0.96 vs $(2/3)^{2/3}$)	literature
7 Growing-resultant ranks + transfers generic rank $C(r+s, r)$ checked to $(4,4)=70$; Semaev S4/S5 specialised ranks 6, 15 = generic ceilings; $N > (H /4)^{(18/37)}$ obstruction (derived); trace-zero kills base-field targets; cover needs $r \not\equiv d$	obstruction
8 Two-product S4 evaluator $R = (F-C)(UL+UR) + (E-B)(VR-VL)$, 2M per pair; exhaustive over F , $p \leq 11$; Gram rank 4 $\Rightarrow$ 2 products optimal; native 2^{61-1} : 1.87x raw / 1.24x monic fused in the indexed regime; loses when endpoints are not reused	evaluator
Claim boundary (all items): no faster NFS, FFS, ECDLP, pairing-target attack or complete index-calculus algorithm is claimed from AVW. Items 2, 5, 8 are stage diagnostics on a shared VM (candidate_id null); items 3, 7 are derived from the premise; items 1, 4, 6 are checked algebra.	

Figure 1. The eight shipped items with their status and the shared claim boundary (source: figures/make_figures.py).

Method

Each item is one Python module writing one JSON record under results/; run_all.sh runs them in order. Exact algebra is checked over $\mathbb{Q}$ (fractions) or $\text{GF}(2^{61-1})$; identities are additionally checked symbolically with SymPy and exhaustively over small fields where the parameter space allows. Wall-time measurements use a fresh process per run (SSS), or a native C program with rotated trial order and medians (S4 evaluator); both ran on a shared 4-vCPU cloud VM without an isolation receipt, so per AGENTS.md "CPU performance isolation gate" every wall-time ratio here is **exploratory**. Operation counts are exact and host-independent.

Both benchmark records embed a machine-readable hardware and execution manifest (hardware_manifest.py, AGENTS.md "Hardware and placement for empirical benchmarks"): CPU vendor, reported model string, CPUID family/model/stepping with the microarchitecture decoded from them (Emerald Rapids, labelled as a CPUID lookup, not independently documented), architecture and feature flags, cores/threads/sockets, per-level cache topology, kernel/OS, virtualization (kvm, no container), cgroup v2 CPU quota and memory limit (max 100000, max), the single exposed NUMA node, allowed CPUs and the fact that no affinity or memory policy was selected, and DIMM technology/speed/channels written as unknown (a VM exposes no verifiable evidence). Each run additionally records process CPU time beside wall time, peak RSS, page faults and context switches, and the host's steal-time fraction from /proc/stat over the run (0.0 for every S4 regime; at most 0.4 % for one SSS run). Nothing is pinned and no core is reserved, so these are execution records, not isolation receipts.

Sources

Source	Role	How it was used
Follow-up report (uploaded PDF, 2026-10-06)	primary	Decision list, premise constants, each item's statement
AVW, arXiv:2610.06783 (as cited by the report; not re-read here)	premise	Exponents $0.063 / 0.437 / 1/18$ and the rank condition, as quoted by the report
Bernstein, <i>How to find smooth parts of integers</i> (2004)	method	Product/remainder-tree smooth parts (batch_smooth.py); recalled, standard
Joux, <i>Faster index calculus for the medium prime case</i> (EUROCRYPT 2013; ePrint 2012/720)	literature	Advanced pinpointing in Kummer extensions and its $L(1/3)$ constants; the stated range $[3^{(-2/3)}, (2/3)^{(2/3)}]$ was recomputed
Hittmeir, Smooth Subsum Search, github.com/sbaresearch/smoothsubsumsearch rev 8dbaf6d3...	benchmark	SSS, bundled pSIQS and SymPy-SIQS drivers fetched at run time (no licence file upstream, so nothing is vendored)
Semaev, <i>Summation polynomials and the discrete logarithm problem on elliptic curves</i> (ePrint 2004/031)	method	S3 and the resultant chain for S4, S5; recalled, standard
Frey (1999) / Gaudry-Hess-Smart (2002)	background	Trace-zero and cover transfers; recalled, only the standard facts $\ker(\text{Frob} - 1) = E(F_q)$ and $\pi_* \pi^* = [d]$ are used, both checked on toy instances

Findings by item

1. Selected-resieve correspondence — checked (selected_resieve.py)

Lemma. Fix a sieve region of N positions and primes $p \leq y$. Let X be $N \times D$ with $D = \sum_{p \leq y} p$ whose row u has a 1 in the residue-state column $(p, u \bmod p)$ for each p , and let Y be $D \times N'$ whose column v carries weight w_p in column (p, ρ) for each root ρ of the sieve polynomial mod p . Then the sieve score of position u against polynomial v is the (u, v) entry of XY , and a *selected resieve* on a set W of positions is exactly a selected product on W . The kernel $K_{\{uv\}} = \sum_p w_p [u \equiv v \bmod p]$ has rank $1 + \sum_p (p - 1) = \theta(D)$.

Checks. Identity sieve scores = selected XY entries: 40 random trials, largest $N \cdot D = 1968$, all equal. Rank of K : prime sets $\{2,3\}$, $\{2,3,5\}$, $\{3,5,7\}$, $\{2,3,5,7\}$, $\{2,3,5,7,11\}$, with N below, at, and above $1 + \sum(p-1)$; measured ranks $\min(N, 1 + \sum(p-1))$ in all 15 cases.

Consequence (derived). $D \sim y^2 / (2 \ln y)$, so $D \leq N^{(1/18)}$ caps the factor base at $y \leq N^{(1/36 + o(1))}$: $y \leq 5$ for $N = 2^{64}$, 29 for 2^{128} , 461 for 2^{256} , 89,317 for 2^{512} . Any exact linear compression of the residue-state representation needs $\theta(D)$ coordinates.

2. Batch-smoothness screen — measured, dominance derived (batch_smooth.py)

Bernstein's product/remainder-tree smooth-part extraction over gmpy2 was checked against trial division on 900 values (all agree) and timed: 25–37 ns per bit of the batch b across $y \in \{2^{12} \dots 2^{20}\}$, $|W| \in \{2^{10} \dots 2^{16}\}$, $|L| \in \{64, 128, 256\}$ bits, i.e. essentially flat in y and $|W|$ as the quasi-linear bound predicts. For an **explicit** candidate set W , the exact test costs $\tilde{O}(\sum \log p + \sum \log |w|)$, while the AVW route costs $N^2 / D^{0.063 + |W| D^{0.437}}$; the derived dominance grid shows the direct method ahead by a factor $D^{0.437}$ inside the preprocessing range and by the query term outside it. The screen is decisive: no explicit candidate set is a thin-product regime.

3. NFS density window — derived (nfs_density_window.py)

Write the collector's useful-entry density as θ and $D = S^\gamma$. AVW beats both a sparse block check and an exact batch test only when $0.437 \gamma < \theta < (0.5 + 0.063) \gamma$; with $\gamma \leq 1/18$ this is $0.024278 < \theta < 0.031278$ (numerically confirmed by brute force over a grid). Standard heuristic densities fall outside: GNFS ($\theta = 1/4$) gives AVW $S^{1.9965}$ against sparse-block $S^{1.7778}$ and exact batch $S^{1.75}$; SNFS ($\theta = 1/2$) gives $S^{1.9965}$ against $S^{1.5278}$ and $S^{1.5}$. Sparse linear algebra stays $S^{(2+o(1))}$ at optimised parameters even with a free first side. The SSS-to-NFS trade-off table (capturing a fraction $S^{(-3 \text{ eps})}$ of relations per pass costs $S^{(2 + 2 \text{ eps})}$ total work) is likewise never below S^2 . Figure: figures/nfs_window.svg.

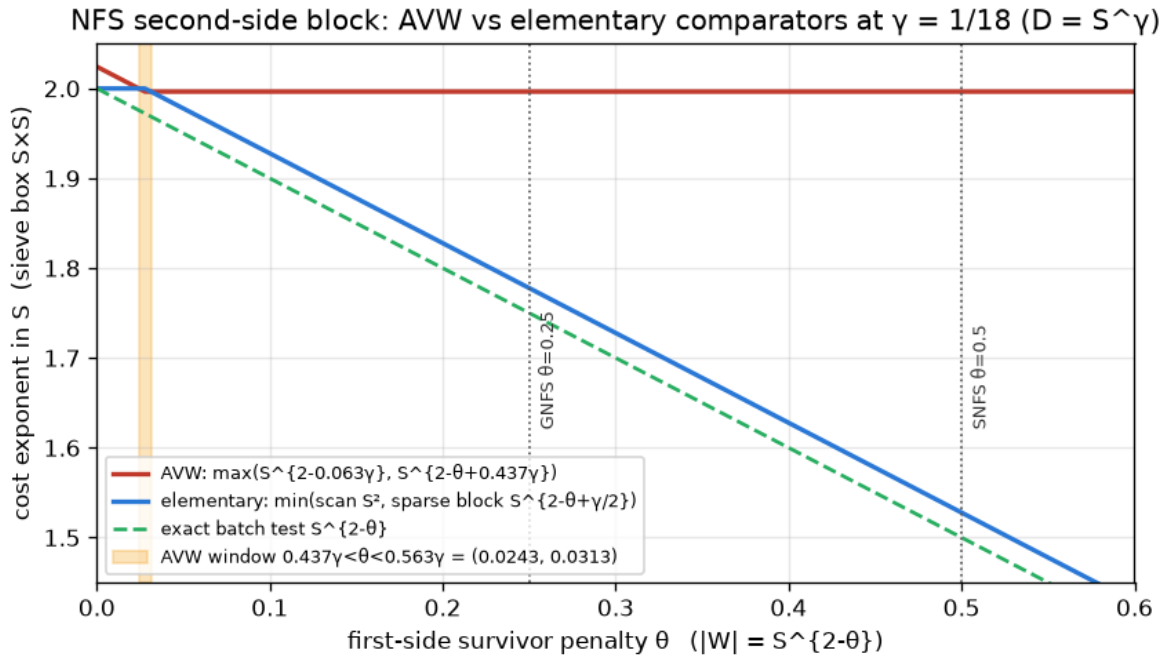


Figure 2. AVW exponent against sparse-block and exact-batch collectors as a function of theta; the window $0.437\gamma < \theta < 0.563\gamma$ at $\gamma = 1/18$ and the GNFS/SNFS inputs (source: `nfs_density_window.py`).

4. Dyadic heavy-cell reporter — checked construction, parameter-screened (`dyadic_reporter.py`)

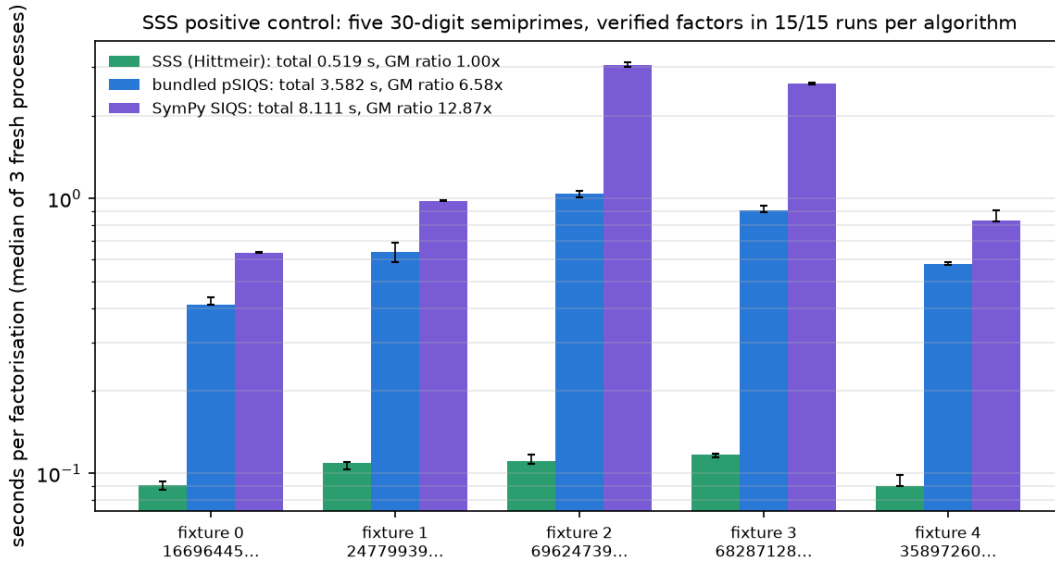
Adding dyadic aggregates as extra rows and columns and splitting every rectangle whose sum reaches τ reports every cell with score $\geq \tau$ exactly for nonnegative scores, using at most $1 + 4 (M / \tau) \log_2 N$ queries (M = total mass). Checked on 25 random instances and 16 sieve-like instances ($y = 47$, $N \in \{16 \dots 512\}$, thresholds $u \cdot \log y$, $u \in \{1, 2, 4, 8\}$): the bound held in every case, and the reporter used about 1.33 queries per heavy cell whenever heavy cells existed. **Screen (derived)**. In an NFS log sieve $M = N^{(2+o(1))} \log y$ and $\tau = u \log y$ with $u = N^{o(1)}$, so $M / \tau = N^{(2 - o(1))}$: the reporter never beats a plain scan of the cells; it would need $u \gg D^{0.437}$.

5. SSS five-fixtured positive control — measured (`sss_benchmark.py`, `sss_runner.py`)

Five 30-digit semiprimes ($p \sim U[1e14, 5e14]$, $q \sim U[5e14, 2.5e15]$, seed 20261006; values in `results/sss/fixtures.json`) were factored by Hittmeir's SSS, the bundled pSIQS, and the bundled SymPy SIQS driver, each in a fresh process, three repetitions, order rotated, import time excluded. All 45 runs returned verified proper factors.

Algorithm	verified	total of per-fixtured medians (s)	geometric-mean ratio to SSS
SSS	15/15	0.519	1.00
bundled pSIQS	15/15	3.582	6.58x
SymPy SIQS	15/15	8.111	12.87x

Per-fixtured medians (s): SSS 0.091 / 0.110 / 0.111 / 0.118 / 0.090; pSIQS 0.412 / 0.639 / 1.046 / 0.909 / 0.576; SymPy SIQS 0.634 / 0.982 / 3.047 / 2.619 / 0.830. Process CPU time equals wall time to within 1 ms on every run (single-threaded, no waiting); peak RSS 56 MB (SSS, pSIQS) and 64 MB (SymPy SIQS). Versions: Python 3.12.3, gmpy2 2.3.2, sympy 1.14.0, numpy 2.4.4. This is a positive control that the harness measures a real constant-factor gain where one is known to exist; it says nothing about AVW, and the wall times are exploratory (shared VM). Figure: `figures/sss_benchmark.svg`. Raw per-run records: `results/sss/run_*.json`.



Whiskers: min/max of 3 repetitions. Upstream rev 8dbaf6d...; python 3.12.3, gmpy2 2.3.2, sympy 1.14.0; shared 4-vCPU VM, no isolation receipt.

Figure 3. Per-fixture medians of three fresh-process runs for SSS, bundled pSIQS and SymPy SIQS; whiskers are min/max; log scale (source: figures/make_figures.py from results/ssss/summary.json).

6. Joux advanced pinpointing — checked (pinpointing.py)

The closest established asymptotic index-calculus speedup from a collision-style join is Joux's advanced pinpointing in Kummer extensions F_{q^n} , $n = d_1 d_2 - 1$, where relations are produced by bucketing $x + a$ and $y + b$ on an n -th-power residue label instead of scanning all pairs. A toy instance ($q = 311$, $d_1 = 2$, $d_2 = 3$, $n = 5$, $\alpha = 2$) reproduced the join: for labels $\lambda \in \{2, 3, 5, 7\}$ the lists had sizes $|L_A| = 14\text{--}18$, $|L_B| = 48\text{--}55$, the join produced 116–198 relations each (matching the expected $|L_A||L_B|/n$ to within 3 %), bucket-join work 180–271 against an all-pairs scan of 583–990, and all 620 relations were verified by exact arithmetic in F_{q^5} ; $\lambda = 1$ is degenerate ($|L_A| = 0$). Cast as a selected product the join is a one-hot inner product of dimension n with separation rank 1, which fails the AVW rank condition ($1 > 5^{0.437} = 2.02$ is false): hashing, not thin products, is what makes it fast.

$L(1/3)$ constants recomputed (α the factor-base exponent): classical sieving $\alpha + 2/(3 \sqrt{\alpha})$, pinpointing $\max(\alpha, 2/(3 \sqrt{\alpha}))$, linear algebra 2α ; pinpointing improves the total exactly for $3^{-2/3} \leq \alpha < (2/3)^{2/3}$, i.e. $0.4807 \leq \alpha < 0.7631$, from 1.4422 to 0.9615 at the lower end. The source's " $(2/3)^{2/3} \approx 0.96$ " is a misprint for $2 \cdot 3^{-2/3} = 0.9615$; $(2/3)^{2/3}$ itself is 0.7631. Figure: figures/pinpointing_constants.svg.

Medium-prime index calculus: relation collection and linear algebra constants (Joux 2)

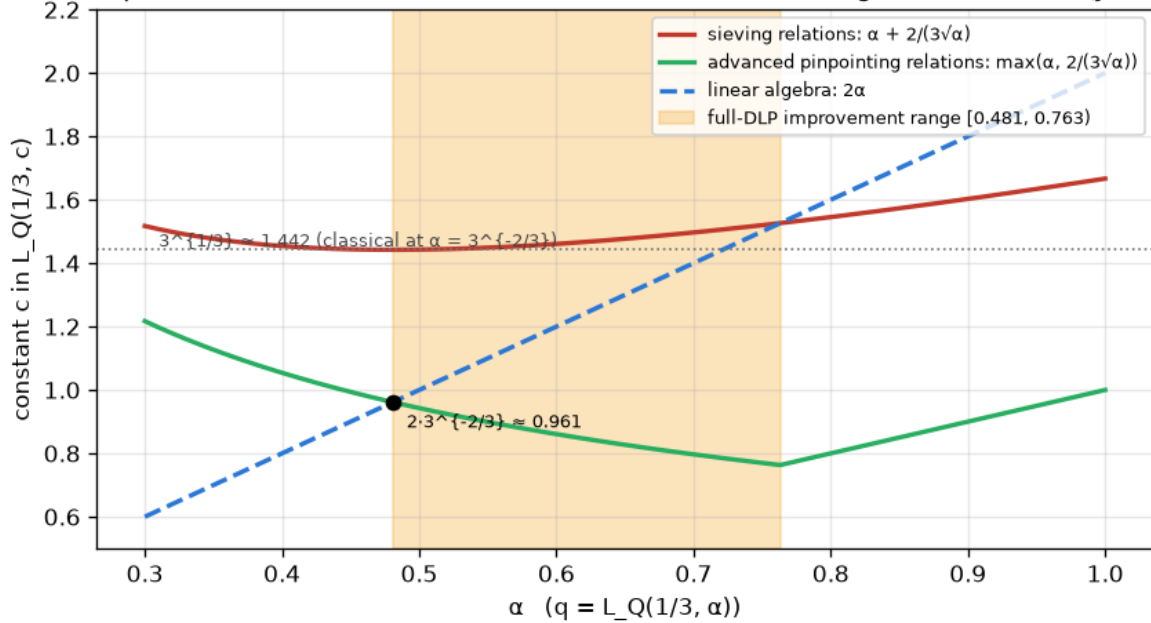


Figure 4. $L(1/3)$ constants: classical sieving, pinpointing relation collection and linear algebra against the factor-base exponent α ; improvement range marked (source: pinpointing.py).

7. Growing-resultant ranks and typed transfer checks — checked / derived

(resultant_ranks.py, transfer_checks.py)

Generic rank (checked). For children of degrees r, s in the glue variable, the flattening of $\text{Res}_z(f, g)$ in $(\text{coefficients of } f) \times (\text{coefficients of } g)$ has rank $C(r+s, r)$. Checked exactly for $(r, s) \in \{(1,1), (1,2), (2,2), (1,3), (2,3), (2,4), (3,3), (4,4)\}$ by expanding the Sylvester determinant as a signed monomial sum over all permutations ($8! = 40,320$ for $(4,4)$, rank $70 = C(8,4)$). Balanced ceilings: $m = 4 \rightarrow 6, 5 \rightarrow 15, 6 \rightarrow 70, 7 \rightarrow 495, 8 \rightarrow 12,870, 10 \rightarrow 601,080,390, 14 \rightarrow 2^{124.17}$.

****Semaev specialisation (measured, exact mod $2^{61} - 1$).** On $y^2 = x^3 + 2x + 3$: $S_4 = \text{Res}(S_3, S_3)$ flattened $(x_1, x_2) \mid (x_3, x_4)$ has rank 6 = generic ceiling; $S_5 = \text{Res}(S_3, S_4)$ flattened $(x_1, x_2) \mid (x_3, x_4, x_5)$ has rank 15 = generic ceiling; one-versus-rest ranks are 3, 5, 9 = $2^{(m-2)} + 1$ for $m = 3, 4, 5$. Ranks were obtained from evaluation matrices with more random points than monomials on each side, and the S_4 values were cross-checked against the fully symbolic expansion (439 terms, degree 4 per variable, agreement at 20 random points). **No rank collapse** appears on this curve; a pathological collapse would have to be demonstrated.

Parameter obstruction (derived). With one-versus-rest rank $2^{(m-2)} + 1 \leq D \leq N^{(1/18)}$ and $(2B)^m \geq |H|$ for constant success, $N > (|H|/4)^{(18/37)}$ and AVW preprocessing $\geq N^{(2 - 0.063/18)} > (|H|/4)^{0.9714}$, far above generic ρ at $|H|^{(1/2)}$. Large rank is not a thin-product regime in any case: lifted factor-base points admit group-sum hashing and x -only children admit intermediate-root hashing after factorisation.

Trace-zero transfer (checked). $q = 1009, n = 3, E: y^2 = x^3 + 2x + 3, \#E(F_q) = 2^2 \cdot 3 \cdot 89 = 1068, \#E(F_{q^3}) = 2^2 \cdot 3^3 \cdot 13 \cdot 89 \cdot 8221$. With $\psi = \text{Frob} - 1$: all 10 random base-field points map to 0; all 10 random extension points have $\text{Tr}(P) \in E(F_q)$ and $\text{Tr}(\psi(P)) = 0$; on the subgroup H of order $r = 8221$ (coprime to $\#E(F_q)$) ψ is linear and the log of $\psi([a]G)$ to base $\psi(G)$ equals a for 5 random a . **Typed rule:** a base-field target is annihilated; only extension-field targets of order prime to $\#E(F_q)$ transfer.

Cover transfer (derived; one instance checked). For a cover $\pi: C \rightarrow E$ of degree $d, \pi^* \pi = [d]$, so $E[r]$ pulls back injectively iff $r \nmid d$, with recovery scalar $d^{(-1) \bmod r}$. The Weil-restriction instance (inclusion $E(F_q) \rightarrow E(F_{q^3})$, trace as push-forward, $d = 3, r = 89$) confirmed $\text{Tr} = [3]$ on base points and recovered the planted log after push-forward. No genus- g cover of any target curve was constructed; the rule is a certificate, not an attack. Neither transfer changes the rank or density of a relation search's inner product.

8. Two-product S4 evaluator — checked identity, measured constant

(s4_two_product.py, s4_bench.c, s4_bench.py)

For monic $f = z^2 + Bz + C$, $g = z^2 + Ez + F$,

$\text{Res}_z(f, g) = (F - C)(UL + UR) + (E - B)(VR - VL)$,
 $UL = B^2 - C$, $VL = B \cdot C$ (left endpoint), $UR = F - E^2$, $VR = E \cdot F$ (right endpoint),

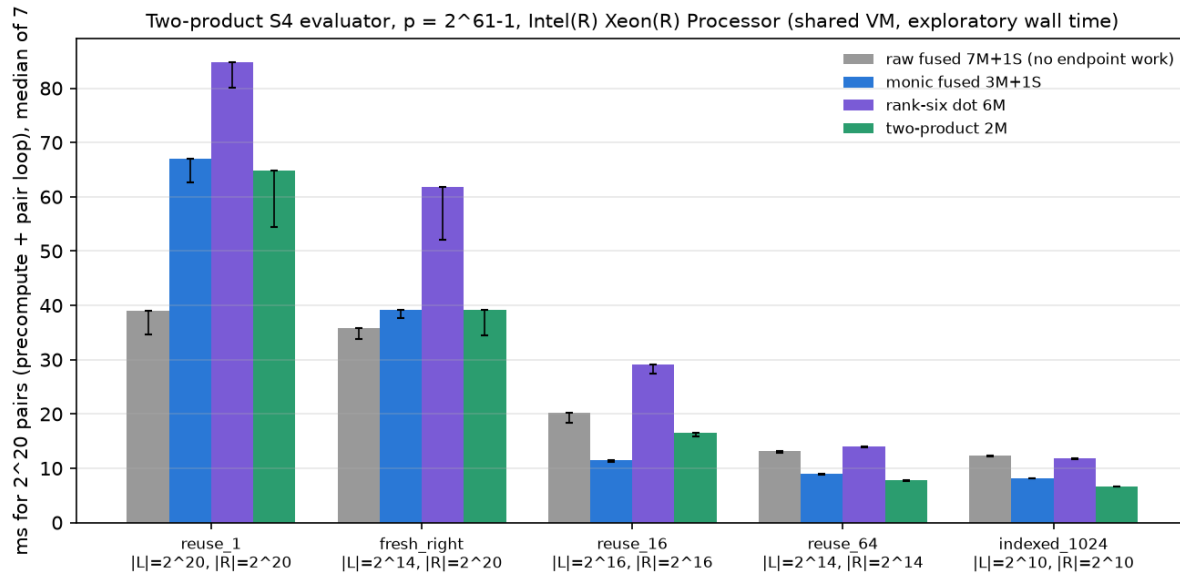
so an $S4 = 0$ test on a candidate pair costs **2 multiplications** once each endpoint's four features are stored (projective resultant = $a_2^2 b_2^2 R$; $a_2 = (x_1 - x_2)^2 = 0$ is the doubling partition and never enters the pair loop). Checked symbolically, and exhaustively over F_p for $p \in \{2, 3, 5, 7, 11\}$ (17,764 tuples, 0 mismatches).

Optimality (checked): the Gram matrix of R in the eight stored features has rank 4, so at least two products of linear forms are needed; the left-x-right cross block also has rank 4, so a plain dot product needs at least 4 multiplications; the full $(B, C) \parallel (E, F)$ flattening has rank $6 = C(4, 2)$.

Native benchmark over $p = 2^{61} - 1$ with real $S3$ coefficients from random points on a random curve, 2^{20} pairs per regime, 7 rotated trials, precomputation (Montgomery batch inversion) charged, medians:

regime	sizes of L, R	raw fused 7M+1S	monic fused 3M+1S	rank-six dot 6M	two-product 2M	raw/2P	monic/2P
reuse 1 (loss case)	$2^{20}, 2^{20}$	39.0 ms	67.0 ms	84.9 ms	64.9 ms	0.60	1.03
fresh right (loss case)	$2^{14}, 2^{20}$	35.8 ms	39.1 ms	61.8 ms	39.2 ms	0.91	1.00
reuse 16	$2^{16}, 2^{16}$	20.2 ms	11.6 ms	29.1 ms	16.6 ms	1.22	0.70
reuse 64	$2^{14}, 2^{14}$	13.2 ms	9.0 ms	14.0 ms	7.7 ms	1.71	1.16
indexed 1024 x 1024	$2^{10}, 2^{10}$	12.3 ms	8.2 ms	11.8 ms	6.6 ms	1.87	1.24

All four evaluators agreed on every pair in every regime (checksums and pairwise comparison). The crossover is where endpoint reuse pays for the feature precomputation: with no reuse the raw evaluator wins; in the cache-resident indexed regime the two-product loop runs at 6.3 ns per pair against 7.8 (monic fused) and 11.7 (raw). Process CPU time equals wall time in every regime (0.30–2.16 s per regime, single thread), peak RSS 17–264 MB depending on the endpoint tables, steal fraction 0.0 throughout. The reuse 16 row shows the gain is memory-bound at that size (32-byte features versus 16-byte monic pairs streamed from a 2 MB table), so the arithmetic saving does not always convert to time. **Claim boundary:** this lowers the constant of the pair-enumeration step of a four-summand decomposition oracle when $|W| > |L| + |R|$ -scale reuse exists; it changes neither the number of pairs visited, the relation yield, nor the linear algebra, is not a thin-product instance (rank 6, every pair visited), and establishes no IC or ECDLP speedup. candidate_id: null. Figure: figures/s4_bench.svg.



Bars: median total; whisker down to the minimum trial. All four evaluators agree on every pair in every regime (checksums). Stage diagnostic; candidate_id null.

Figure 5. Wall time for 2^{20} S4 = 0 tests per regime and evaluator over $p = 2^{61} - 1$, medians of 7 rotated trials with the minimum as whisker; shared VM, exploratory (source: figures/make_figures.py from results/s4_bench.json).

Negative and inconclusive findings

- No explicit candidate set, NFS collector, FFS sieve, or Semaev decomposition visited here lands inside the AVW window; each is either below the density floor (theta too large, items 2–4), rank-deficient for the premise (item 6), or parameter-obstructed (item 7).
- The Semaev specialised ranks show no collapse below the generic ceiling on the one curve tested (S4: 6, S5: 15). This is evidence on one curve over q reduced mod one prime, not a theorem.
- The two-product evaluator loses to the raw evaluator when endpoints are not reused, and to monic fused in one memory-bound regime.
- The upstream SSS repository has no licence file; it is fetched at run time (SSS_UPSTREAM_DIR or a temporary clone at the pinned revision), so a network-free rerun needs a local clone.

Graph set checked

No curve, isogeny, conductor, volcano level, or scalar rule changed, so the canonical graphs were checked and left unchanged: experiments/volcano-ic/figures/ (census, ecdlp, tau), experiments/ic-candidate-catalog/curves.yaml and its isogeny-route records, and paper/ figures. The new figures live only in this directory. The toy curves used in items 7 and 8 ($y^2 = x^3 + 2x + 3$ over F_{1009} , F_{1009^3} , and random curves over $F_{2^{61}-1}$) are scaffolding for algebraic checks, not finalized curves, and are not registered.

Reproduction

```
cd experiments/speedup-search-followup-20261006
./run_all.sh          # ~1 min plus the SSS benchmark (~2 min, needs git/network or SSS_UPSTREAM_DIR)
python3 build_pdf.py  # regenerates speedup_search_followup.pdf
```

Dependencies: Python 3.12, sympy, numpy, gmpy2, matplotlib, reportlab, gcc. Every result file under results/ is written by the module of the same name; results/s4_bench.json and results/sss/summary.json carry the hardware and execution manifest, the source hash of s4_bench.c, and the per-run process/steal records.